

Unit Vector Calculation in Python

Meenu S.

January 16, 2026

Unit Vector Calculation in Python

The following Python program allows the user to input a vector of arbitrary dimension and computes its unit vector in both the same and opposite directions.

Code with Explanations

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from mpl_toolkits.mplot3d import Axes3D # Required for 3D
4     plots
5 def get_dimension():
6     """
7     Asks the user for the dimension of the vector.
8     Continues prompting until a valid positive integer is
9     entered.
10    """
11    while True:
12        try:
13            dim_str = input("Enter the dimension of the
14                            vector: ")
15            dim = int(dim_str)
16            if dim > 0:
17                return dim
18            else:
19                print("Dimension must be a positive integer.
20                      ")
21        except ValueError:
22            print("Invalid input. Please enter a single
23                  integer.")
24 def get_vector(dim, vector_name="v"):
```

```

20     """
21     Gets a valid vector from the user for a given dimension.
22     """
23     while True:
24         # Generate an example for the prompt based on the
25         dimension
26         example = ', '.join(str(i) for i in range(1, dim + 1))
27         prompt = f"Enter the {dim} elements for vector {
28             vector_name}, separated by commas (e.g., '{
29             example}'): "
30         vector_str_input = input(prompt)
31         vector_str_list = vector_str_input.split(',')
32         if len(vector_str_list) != dim:
33             print(f"Error: Incorrect number of elements.
34                 Please enter exactly {dim} elements.")
35             continue
36         try:
37             # Convert string elements to floats and create a
38             numpy array
39             v = np.array([float(x.strip()) for x in
40                 vector_str_list])
41             return v
42         except ValueError:
43             print("Invalid input. Please ensure all elements
44                 are numbers.")
45     def format_vector_for_display(vector):
46         """
47         Formats a numpy vector as a string with comma-separated
48         elements.
49         """
50         return "(" + ", ".join([f"{x:.4f}" for x in vector]) + "
51             )"
52     def main():
53         """
54         Main function to find unit vectors in the same and
55         opposite directions.
56         """
57         # 1) Ask for the dimension
58         dim = get_dimension()
59
60         # 2) Get vector from user
61         v = get_vector(dim)

```

```

52 print(f"\nOriginal vector v: {format_vector_for_display(
    v)}")
53 # Calculate the norm (magnitude) of the vector v
54 norm_v = np.linalg.norm(v)
55 if norm_v == 0:
56     print("\nCannot calculate unit vectors for a zero
        vector.")
57     return
58 # 3) Compute unit vectors
59 result1 = v / norm_v          # Unit vector in same
    direction
60 result2 = -result1           # Unit vector in opposite
    direction
61 # 4) Display the results
62 print("\n--- Unit Vector Results ---")
63 print(f"The unit vector of v {format_vector_for_display(
    v)}:")
64 print(f"  In the direction of v: {
    format_vector_for_display(result1)}")
65 print(f"  In the opposite direction of v : {
    format_vector_for_display(result2)}")
66 # 5) Plot the vectors if dimension is 2 or 3
67 if dim == 2:
68     plt.figure(figsize=(8, 8))
69     ax = plt.gca()
70     # Plot original vector v
71     ax.quiver(0, 0, v[0], v[1], angles='xy', scale_units
        ='xy', scale=1, color='blue', label='Original
        Vector v', width=0.005, headwidth=5, headlength
        =8)
72     # Plot unit vector in same direction
73     ax.quiver(0, 0, result1[0], result1[1], angles='xy',
        scale_units='xy', scale=1, color='green', label=
        'Unit Vector (same direction)', width=0.005,
        headwidth=5, headlength=8)
74     # Plot unit vector in opposite direction
75     ax.quiver(0, 0, result2[0], result2[1], angles='xy',
        scale_units='xy', scale=1, color='red', label=
        'Unit Vector (opposite direction)', width=0.005,
        headwidth=5, headlength=8)
76     # Set limits and labels
77     max_val = max(abs(v).max(), 1.5) # Ensure at least
        unit vectors are visible

```

```

78     ax.set_xlim([-max_val - 0.5, max_val + 0.5])
79     ax.set_ylim([-max_val - 0.5, max_val + 0.5])
80     ax.set_aspect('equal', adjustable='box')
81     ax.axhline(0, color='gray', linewidth=0.5)
82     ax.axvline(0, color='gray', linewidth=0.5)
83     plt.xlabel('X-axis')
84     plt.ylabel('Y-axis')
85     plt.title('Vector and Unit Vectors in 2D')
86     plt.legend()
87     plt.grid(True)
88     plt.show()
89 elif dim == 3:
90     fig = plt.figure(figsize=(10, 10))
91     ax = fig.add_subplot(111, projection='3d')
92     # Plot original vector v
93     ax.quiver(0, 0, 0, v[0], v[1], v[2], color='blue',
94              label='Original Vector v', arrow_length_ratio
95              =0.1)
96     # Plot unit vector in same direction
97     ax.quiver(0, 0, 0, result1[0], result1[1], result1
98              [2], color='green', label='Unit Vector (same
99              direction)', arrow_length_ratio=0.1)
100     # Plot unit vector in opposite direction
101     ax.quiver(0, 0, 0, result2[0], result2[1], result2
102              [2], color='red', label='Unit Vector (opposite
103              direction)', arrow_length_ratio=0.1)
104     # Set limits and labels
105     max_val = max(abs(v).max(), 1.5) # Ensure at least
106     unit vectors are visible
107     ax.set_xlim([-max_val - 0.5, max_val + 0.5])
108     ax.set_ylim([-max_val - 0.5, max_val + 0.5])
109     ax.set_zlim([-max_val - 0.5, max_val + 0.5])
110     ax.set_xlabel('X-axis')
111     ax.set_ylabel('Y-axis')
112     ax.set_zlabel('Z-axis')
113     plt.title('Vector and Unit Vectors in 3D')
114     plt.legend()
115     plt.grid(True)
116     plt.show()
117 else:
118     print("\nPlotting is supported only for 2D and 3D
119           vectors.")
120 if __name__ == "__main__":

```

Listing 1: Python code to compute unit vectors

OUTPUT

Enter the dimension of the vector: 2

Enter the 2 elements for vector v, separated by commas (e.g., '1,2'): -5,12

Original vector v: (-5.0000, 12.0000)

— Unit Vector Results —

The unit vector of v (-5.0000, 12.0000):

In the direction of v: (-0.3846, 0.9231)

In the opposite direction of v : (0.3846, -0.9231)

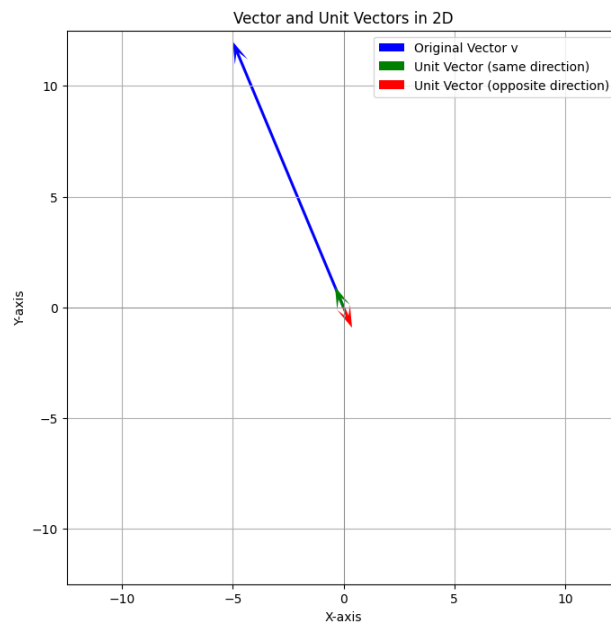


Figure 1: 2D vector and its unit vectors in same and opposite directions

Explanation: The `main` function orchestrates the program:

1. Prompts the user for the vector dimension.
2. Accepts the vector input from the user.
3. Computes the magnitude (norm) of the vector.

4. Calculates the unit vector in both the same and opposite directions.
5. Prints the results in a clear, formatted manner.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from mpl_toolkits.mplot3d import Axes3D # Required for 3D
4     plots
5 def get_dimension():
6     """
7     Asks the user for the dimension of the vector.
8     Continues prompting until a valid positive integer is
9     entered.
10    """
11    while True:
12        try:
13            dim_str = input("Enter the dimension of the
14                            vector: ")
15            dim = int(dim_str)
16            if dim > 0:
17                return dim
18            else:
19                print("Dimension must be a positive integer.
20                      ")
21        except ValueError:
22            print("Invalid input. Please enter a single
23                  integer.")
24 def get_vector(dim, vector_name="v"):
25     """
26     Gets a valid vector from the user for a given dimension.
27     """
28     while True:
29         # Generate an example for the prompt based on the
30         # dimension
31         example = ', '.join(str(i) for i in range(1, dim + 1))
32         prompt = f"Enter the {dim} elements for vector {
33                  vector_name}, separated by commas (e.g., '{
34                  example}'): "
35         vector_str_input = input(prompt)
36         vector_str_list = vector_str_input.split(',')
37         if len(vector_str_list) != dim:
38             print(f"Error: Incorrect number of elements.
39                   Please enter exactly {dim} elements.")
```

```

31         continue
32     try:
33         # Convert string elements to floats and create a
34         numpy array
35         v = np.array([float(x.strip()) for x in
36                        vector_str_list])
37         return v
38     except ValueError:
39         print("Invalid input. Please ensure all elements
40               are numbers.")
41
42 def format_vector_for_display(vector):
43     """
44     Formats a numpy vector as a string with comma-separated
45     elements.
46     """
47     return "(" + ", ".join([f"{x:.4f}" for x in vector]) + "
48         )"
49
50 def get_positive_float_input(prompt):
51     """
52     Asks the user for a positive float value.
53     """
54     while True:
55         try:
56             value_str = input(prompt)
57             value = float(value_str)
58             if value > 0:
59                 return value
60             else:
61                 print("Value must be positive.")
62         except ValueError:
63             print("Invalid input. Please enter a valid
64                   number.")
65
66 def main():
67     """
68     Main function to find a vector of a given length in the
69     direction of another vector.
70     """
71     # 1) Ask for the dimension
72     dim = get_dimension()
73     # 2) Get the reference vector from the user
74     ref_vector = get_vector(dim, vector_name="reference")
75     print(f"\nReference vector: {format_vector_for_display(
76           ref_vector)}")

```

```

66     # 3) Get the desired length from the user
67     desired_length = get_positive_float_input("Enter the
        desired length for the new vector: ")
68     # Calculate the norm (magnitude) of the reference
        vector
69     norm_ref_vector = np.linalg.norm(ref_vector)
70     if norm_ref_vector == 0:
71         print("\nCannot determine direction from a zero
            reference vector.")
72         return
73     # 4) Calculate the unit vector of the reference vector
74     unit_vector = ref_vector / norm_ref_vector
75     # 5) Multiply the unit vector by the desired length
76     result_vector = unit_vector * desired_length
77     # 6) Display the results
78     print("\n--- Result --- ")
79     print(f"Desired length: {desired_length:.4f}")
80     print(f"Vector of length {desired_length:.4f} in the
        direction of {format_vector_for_display(ref_vector)}:
        {format_vector_for_display(result_vector)}")
81     # 7) Plot the vectors if dimension is 2 or 3
82     if dim == 2:
83         plt.figure(figsize=(8, 8))
84         ax = plt.gca()
85         # Plot reference vector
86         ax.quiver(0, 0, ref_vector[0], ref_vector[1], angles
            = 'xy', scale_units='xy', scale=1, color='blue',
            label='Reference Vector', width=0.005, headwidth
            =5, headlength=8)
87         # Plot result vector
88         ax.quiver(0, 0, result_vector[0], result_vector[1],
            angles='xy', scale_units='xy', scale=1, color='
            green', label=f'Vector of length {desired_length
            :.2f}', width=0.005, headwidth=5, headlength=8)
89         # Set limits and labels
90         max_abs_ref = np.max(np.abs(ref_vector))
91         max_abs_result = np.max(np.abs(result_vector))
92         max_val = max(max_abs_ref, max_abs_result, 1.5) #
            Ensure at least unit vectors are visible
93         ax.set_xlim([-max_val - 0.5, max_val + 0.5])
94         ax.set_ylim([-max_val - 0.5, max_val + 0.5])
95         ax.set_aspect('equal', adjustable='box')
96         ax.axhline(0, color='gray', linewidth=0.5)

```

```

97     ax.axvline(0, color='gray', linewidth=0.5)
98     plt.xlabel('X-axis')
99     plt.ylabel('Y-axis')
100    plt.title(f'Vector and New Vector of Length {
        desired_length:.2f} in 2D')
101    plt.legend()
102    plt.grid(True)
103    plt.show()
104    elif dim == 3:
105        fig = plt.figure(figsize=(10, 10))
106        ax = fig.add_subplot(111, projection='3d')
107        # Plot reference vector
108        ax.quiver(0, 0, 0, ref_vector[0], ref_vector[1],
            ref_vector[2], color='blue', label='Reference
            Vector', arrow_length_ratio=0.1)
109        # Plot result vector
110        ax.quiver(0, 0, 0, result_vector[0], result_vector
            [1], result_vector[2], color='green', label=f'
            Vector of length {desired_length:.2f}',
            arrow_length_ratio=0.1)
111        # Set limits and labels
112        max_abs_ref = np.max(np.abs(ref_vector))
113        max_abs_result = np.max(np.abs(result_vector))
114        max_val = max(max_abs_ref, max_abs_result, 1.5) #
            Ensure at least unit vectors are visible
115        ax.set_xlim([-max_val - 0.5, max_val + 0.5])
116        ax.set_ylim([-max_val - 0.5, max_val + 0.5])
117        ax.set_zlim([-max_val - 0.5, max_val + 0.5])
118        ax.set_xlabel('X-axis')
119        ax.set_ylabel('Y-axis')
120        ax.set_zlabel('Z-axis')
121        plt.title(f'Vector and New Vector of Length {
            desired_length:.2f} in 3D')
122        plt.legend()
123        plt.grid(True)
124        plt.show()
125    else:
126        print("\nPlotting is supported only for 2D and 3D
            vectors.")
127    if __name__ == "__main__":
128        main()

```

Listing 2: Python code

OUTPUT

Enter the dimension of the vector: 2

Enter the 2 elements for vector reference, separated by commas (e.g., '1,2'): 1,1

Reference vector: (1.0000, 1.0000)

Enter the desired length for the new vector: 4

— Result —

Desired length: 4.0000

Vector of length 4.0000 in the direction of (1.0000, 1.0000): (2.8284, 2.8284)

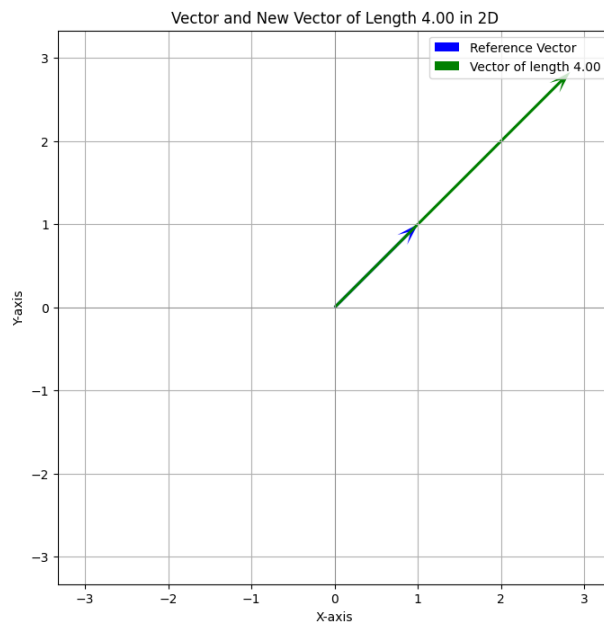


Figure 2: 2D representation